AI Customer Service Agent for Empório da Música

Technical case submission | Artefact — AI Engineer

Candidate: Gabriel Pereira Santos
Date: November 3, 2026
Repository: https://github.com/gabirusky/teste-emporio-agent

1. Project Overview

Empório da Música is a fictional musical instrument store in Campo Grande, Brazil. Its team receives recurring questions about product availability, prices, order status, delivery, payment methods, and return policies. The goal of this project is to prototype a Python text agent that handles those requests while keeping business facts grounded in the supplied data.

The solution follows one principle: **the language model understands and communicates; application tools retrieve the facts.** Prices, stock, orders, promotions, and policy rules are never guessed by the model.

The agent adopts the store persona from the policy manual: friendly, welcoming, informal yet professional, like a friend who understands music. The customer-facing language follows the customer's latest message: Brazilian Portuguese for Portuguese and English for English.

2. Solution Architecture

The implementation is a hybrid architecture combining LLM function calling, structured CSV lookups, and lightweight lexical/fuzzy retrieval over the policy PDF. The agent is exposed through a terminal CLI and a local browser chat.

```
Customer message (CLI or web)
        |
        v
MusicStoreAgent + in-memory history
        |
        v
LLM with function-calling schemas
        |
        +--> search_products ----------> products + promotions + categories
        +--> get_product_details ------> products + promotions
        +--> get_order_status ---------> orders + order items
        +--> search_policies ----------> policy PDF (section-aware RAG)
        +--> redirect_out_of_scope ---> store scope rules
        |
```

```
        v
Grounded answer in the customer's language
```

Main modules:

Module	Responsibility
`agent.py`	System prompt, tool loop, language policy, history, safety fallbacks
`tools.py`	Tool wrappers and OpenAI-compatible function schemas
`data_store.py`	CSV loading, validation, fuzzy search, promotions, orders
`policy_retriever.py`	PDF extraction, section-aware chunks, bilingual query expansion
`cli.py`	Terminal interface with `/reset` and `/exit`
`web.py`	Local browser chat server, per-session history, and JSON endpoints
`config.py`	Environment-backed settings

3. Technical Decisions

Why function calling

The model does not receive the full datasets in the prompt. It selects a targeted Python tool, and the tool reads the authoritative source. This keeps answers anchored in the supplied data, reduces context size, and makes each business fact auditable.

Why retrieval (RAG)

Returns, payments, warranty, shipping, and service rules live in an eight-page PDF. The agent retrieves only the relevant excerpts and keeps the source page number. This avoids putting the entire manual in every prompt and lets the model answer policy questions with references.

Why a hybrid design

Commercial information requires precision. The LLM is effective at understanding intent and writing the response; deterministic tools are better at filtering stock, applying active discounts, and locating policy passages. Separating those responsibilities makes the system easier to test, explain, and improve.

Why lexical/fuzzy RAG instead of embeddings

The knowledge base is a single short PDF. Section-aware chunking plus lexical overlap and RapidFuzz similarity is simple, fast, transparent, and sufficient for this scope. A vector database would add operational complexity without a clear benefit for eight pages. If the corpus grows or semantic recall becomes insufficient, the retrieval layer can be upgraded to embeddings and a vector index.

Model and provider

The client uses the OpenAI Python SDK with a configurable base URL. The submitted configuration was verified with NVIDIA NIM using `nvidia/nemotron-3.5-lightning-30b-a3b`. OpenRouter is supported as an alternative by changing `LLM_BASE_URL`, `LLM_API_KEY`, and `LLM_MODEL`.

The model must support tool/function calling. Credentials are stored only in a local `.env` file, which is excluded from version control.

Conversation history

The current session keeps recent messages in memory and sends the latest 16 messages as context. History is not persisted to disk. This avoids retaining customer conversations in the prototype. A production version could add controlled persistence with retention and LGPD safeguards.

4. Data Processing

The project uses the supplied files:

File	Use
`products.csv`	Catalog, prices, stock, status, specifications
`orders.csv`	Order status, totals, payment, tracking, delivery estimate
`order_items.csv`	Line items in order lookups
`categories.csv`	Official category names
`promotions.csv`	Active promotions and discount percentages
`customers.csv`	Loaded and validated, not used for identity verification
`policies.pdf`	Store policy manual

Key rules:

- A product is available only when `status == active` and `stock_quantity > 0`.
- Only promotions with `is_active` truthy are applied; the highest active discount wins if a product appears more than once.
- The final price is the list price minus the active promotion. Promotions are not cumulative.
- Text matching normalizes accents and uses RapidFuzz for approximate names and descriptions.
- Policy chunks are built from numbered sections, with a fixed-size fallback, and each result keeps its source page.
- English policy questions are expanded to Portuguese terms because the official manual is in Portuguese.

5. Agent Capabilities

Capability	Behavior
Product search	Filters by text, category, availability, and maximum price
Specific product	Returns price, final price, stock, specs, and promotion
Order status	Returns status, items, tracking, and delivery estimate
Policy answers	Retrieves relevant excerpts and cites the page
Out-of-scope	Refuses accessories and unrelated topics, redirects to supported help
Category awareness	Requests for instrument categories absent from the catalog (e.g., flauta doce) return zero results instead of unrelated matches
Interfaces	Terminal CLI and local browser chat with per-session history

Safety behavior:

- The agent does not invent prices, stock, deadlines, promotions, or policy rules.
- It warns that orders use demonstration data and have no customer authentication.
- It does not imply ownership of an order without verification.
- It cannot place orders, reserve stock, process payments, cancel orders, or change records.
- It treats customer messages and retrieved text as data, not instructions.
- It handles provider failures and malformed tool arguments with safe fallbacks.
- It deduplicates repeated tool calls within the same turn.

6. Testing and Evaluation

Automated tests

```
python -m pytest -q
```

Result: 32 passed.

Coverage includes availability rules, price filters, active promotions, zero-stock products, fuzzy lookup, order items and warnings, policy retrieval for returns/hours/scope, written price limits, out-of-scope redirection, tool errors, provider failure, max tool rounds, language directives, order-ownership sanitization, history window, web endpoints, per-session isolation, reset, unsupported instrument categories (zero results without unrelated matches), and tool-call deduplication within a turn.

Live interaction scenarios

The five files in `examples/` are real transcripts from `nvidia/nemotron-3.5-lightning-30b-a3b` via NVIDIA NIM, generated on November 3, 2026.

Scenario	Tool used	Observed result
Acoustic/classical guitars up to R$1,000	`search_products`	Available models with prices and stock
Takamine GD20 price	`get_product_details`	R$ 2.199,00 and 5 units in stock
Online return after changing mind	`search_policies`	7-day window, conditions, refund, page 4
Status of order 8	`get_order_status`	Shipped, tracking code, item, delivery estimate, authentication warning
Guitar picks and cables	`redirect_out_of_scope`	Polite refusal and redirection to instruments

Each transcript records the customer message, the actual tool call, the assistant response, and the observed behavior:

- `examples/01_product_search.md`
- `examples/02_specific_price.md`
- `examples/03_return_policy.md`
- `examples/04_order_status.md`
- `examples/05_out_of_scope.md`

7. Limitations and Future Work

Known limitations:

1. **Static data:** the CSVs are a snapshot, not a live ERP or inventory system.
2. **No order authentication:** `customers.csv` does not provide a login or identity challenge. Real order data must not be exposed without verification.
3. **Lexical RAG:** retrieval does not use embeddings or a vector database.
4. **Promotion coverage:** only the `is_active` flag is authoritative; recurring campaigns are not assumed to be active.
5. **Policy ambiguities:** the manual contains conflicting contact numbers and an email spelling variation. The agent should recommend confirmation rather than choose one.
6. **In-memory history:** conversation state is lost when the process exits.
7. **No side effects:** the agent cannot place orders, process payments, cancel orders, or modify records.
8. **Provider dependency:** reliable tool calling depends on the selected model and endpoint.

Future work:

- Add authentication and authorization for order access.
- Integrate live inventory, orders, and promotion services.
- Add a structured promotion calendar with dates and eligibility rules.
- Evaluate retrieval with a labeled question set; add embeddings/reranking if needed.

- Add human handoff for complaints and ambiguous policy cases.
- Add structured logs for tool selection, latency, errors, and retrieval sources.
- Persist conversation state with retention and LGPD controls if required.

8. Repository, Setup, and AI-Assisted Workflow

Public repository: https://github.com/gabirusky/teste-emporio-agent

Setup:

```
git clone https://github.com/gabirusky/teste-emporio-agent.git
cd teste-emporio-agent

python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -e ".[dev]"

cp .env.example .env
# Set LLM_BASE_URL, LLM_API_KEY, and LLM_MODEL in .env

python -m pytest -q
emporio-agent    # terminal chat
emporio-web --port 8000   # browser chat at http://127.0.0.1:8000 — short command to open the web
```

Verified configuration:

```
LLM_BASE_URL=https://integrate.api.nvidia.com/v1
LLM_API_KEY=your_nvidia_key_here
LLM_MODEL=nvidia/nemotron-3.5-lightning-30b-a3b
```

The `.env` file must remain local and must never be committed.

Web chat interface

The local browser interface uses the Python standard library, so it adds no dependencies. It keeps one conversation per browser session, shows the tools used for each answer, and includes a reset button. It binds to `127.0.0.1` by default; use `--host` and `--port` to change the address. The server has no authentication and is intended for local testing.

AI-assisted development workflow

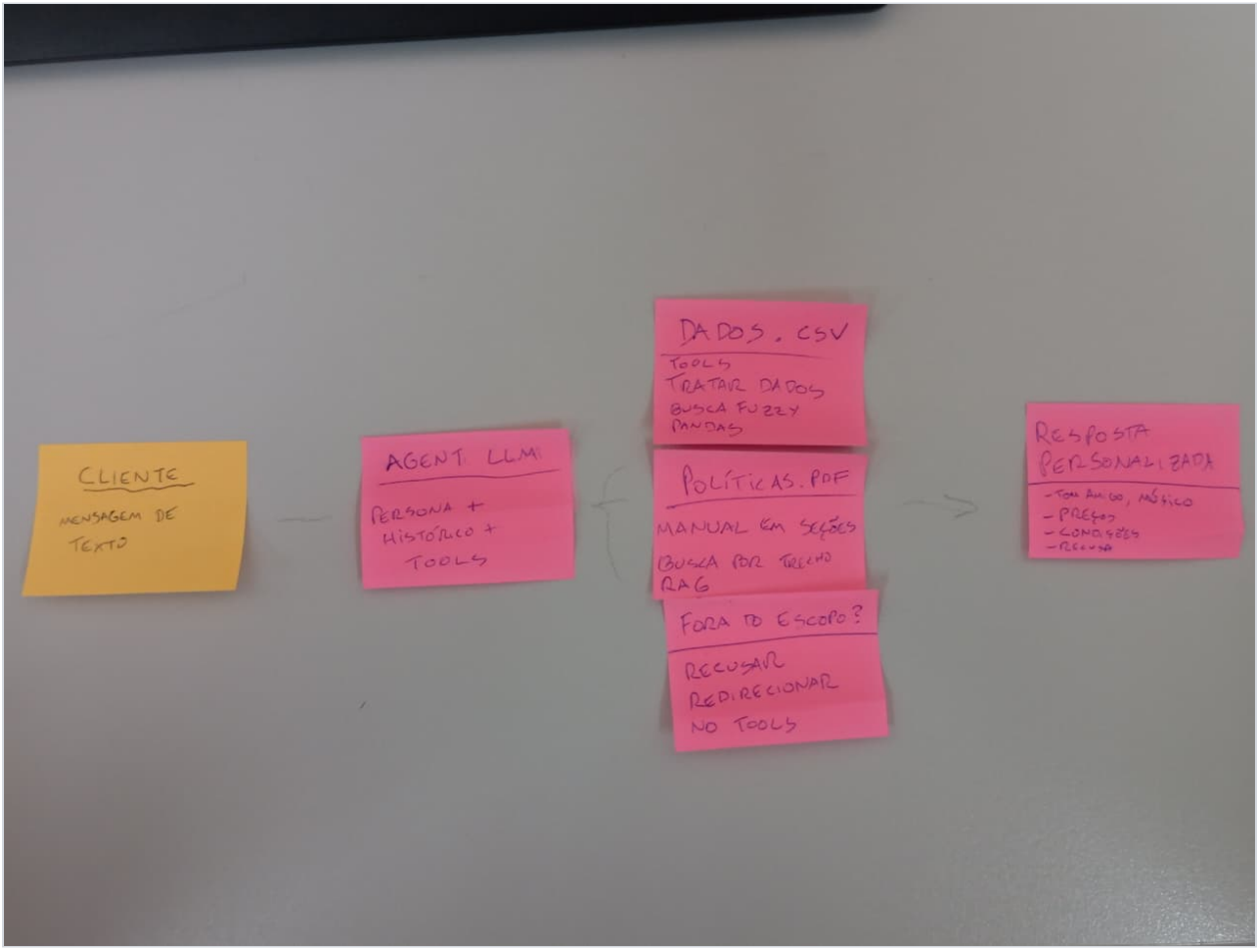
This project was developed with **opencode** using the **DeepSeek V4.1 Flash**, **Nemotron 3.5 Super VL**, and **Step 5 Preview** models as AI coding assistants.

The assistant supported project structuring, Python implementation, debugging, test creation, documentation, and architectural decisions. All generated code and suggestions were reviewed, executed, and validated with the automated test suite and live provider calls. The final architecture,

limitations, and results are documented in the README so they can be explained and defended independently.

Appendix A — Planning Sketch

This hand-drawn sketch was used to plan the agent flow before implementation. It maps the customer message to an LLM agent with persona, history, and tools, then to CSV lookups and policy retrieval, with an out-of-scope branch and a personalized response. The notes are in Portuguese because the store and its manual are in Portuguese.



Planning sketch used to define the tool flow, data sources, and response behavior before implementation.